

Exposé

**Reflection and Self-Directed Activity
in AI-Supported Programming
Education**

Investigating the Impact of Reflective Prompts on Student Engagement
and Problem-Solving in Automated Code Assessment

Supervisor: Johannes Feiner
Submitted by: Paria Nikparsa

March 2026

1 Introduction

In recent years, the use of AI-based code generation tools such as ChatGPT has increased significantly in programming education. These tools can support students in solving programming tasks, explaining concepts, and generating example code. However, their increasing availability also raises concerns regarding students' learning behavior. In particular, excessive reliance on AI-generated solutions may reduce active engagement with programming tasks and hinder the development of independent problem-solving skills.

Completely prohibiting the use of such tools is difficult to enforce in practice and may also be pedagogically questionable, as students are expected to work with AI-supported tools in future professional environments. Instead of banning AI tools, an alternative approach is to encourage students to critically reflect on their learning strategies and their use of external assistance.

One possible approach to foster such reflection is the use of reflective prompts. Reflective prompts are short questions presented to students after completing a programming task. These prompts encourage learners to think about their problem-solving strategies, their learning process, and their use of external tools, including AI systems.

Examples of such prompts include:

- “How did you approach this task? Did you first attempt to solve it independently or did you use external assistance?”
- “What was the most challenging aspect of the task for you?”
- “If you used an AI tool: What did you learn from the generated solution? Could you explain the solution to someone else?”
- “What would you do differently when solving a similar task in the future?”

The goal of these prompts is to increase learners' awareness of their own learning strategies and to encourage more active engagement with programming tasks rather than passive reliance on AI-generated solutions.

To empirically investigate this approach, an Automated Code Assessment (ACA) system will be extended with a reflection module that presents such prompts after task completion. The effectiveness of this intervention will be evaluated through a controlled experiment.

Previous research shows that automated assessment systems in programming education primarily focus on evaluating correctness and efficiency of solutions rather than promoting reflective learning **Messer:2024; Ihantola:2010**. In addition, **Jukiewicz:2024** highlights both the potential and the limitations of AI-based approaches for automated grading. This thesis addresses these limitations by extending an ACA system with a reflection module designed to promote self-directed learning and reflective engagement with programming tasks.

2 Thesis Statement & Problem Description

Thesis statement: Reflective prompts integrated into an automated code assessment system can increase students' self-directed learning behavior and reduce passive reliance on AI-based code generation tools.

Problem: Students increasingly use AI tools such as ChatGPT to generate programming solutions instead of solving tasks independently. While these tools can provide useful support, excessive reliance on them may limit the development of programming skills and problem-solving abilities. Simply banning these tools is difficult to enforce and may conflict with the goal of preparing students for future work environments in which AI assistance is common.

Context: Programming courses often rely on automated code assessment systems to evaluate student submissions at scale. Existing ACA systems such as CodeRunner or Web-CAT mainly focus on assessing correctness through automated tests. However, little attention has been paid to integrating mechanisms that promote reflection or self-regulated learning within such systems.

3 Research Questions and Hypotheses

3.1 Research Question (RQ)

To what extent do reflective prompts about AI tool usage influence students' self-directed learning behavior and independent problem-solving in programming tasks?

3.2 Hypothesis

Reflective activities, such as short prompts about learning strategies and AI usage, increase self-directed activity and reduce passive reliance on AI-based code generation tools.

3.3 Alternatives Considered

When addressing the problem of passive reliance on AI tools in programming education, several approaches can be considered:

- **Banning AI tools:** This approach would attempt to prevent the use of tools such as ChatGPT during programming tasks. However, such restrictions are difficult to enforce in practice and may be pedagogically questionable, given the increasing importance of AI tools in professional software development.
- **No intervention:** Maintaining the current situation would allow students to use AI tools without any additional guidance. In this case, passive reliance on AI-generated solutions may continue to increase.
- **Reflection module:** This approach encourages students to reflect on their learning strategies and their use of external tools without restricting access to AI systems. By prompting students to consider how and why they use AI assistance,

the approach supports self-regulated learning and more active engagement with programming tasks.

Based on these considerations, the reflection module is adopted as the intervention in this thesis. The module will be implemented and evaluated within the ACA system (see Section 4).

4 Method

4.1 Design

Quasi-experimental study with two groups:

- **Control group:** Standard ACA (no reflection module)
- **Experimental group:** ACA with reflection module

4.2 Reflection Module and Exemplary Items

A reflection module was integrated into the ACA to support learners' metacognitive processes and to collect data on their use of external tools during programming tasks.

After each task, students are prompted with a short reflection consisting of:

- A reflective question encouraging metacognitive thinking (e.g., "What did you learn in this exercise?").
- Self-report items measuring the extent of tool usage and individual contribution, implemented as Likert-scale questions or sliders.
- Storage of responses within the system database to enable later quantitative and qualitative analysis.

Exemplary Self-Report Items

The following items illustrate how students report their engagement and use of AI tools:

Individual contribution

"I wrote the code primarily myself."
(1 = Strongly disagree ... 5 = Strongly agree)

AI tool usage

"I used AI tools (e.g., ChatGPT) while working on this task."
(Never / Sometimes / Often / Always)

Understanding of generated code

“If I used AI-generated code, I ensured that I understood it before submitting the solution.”

(1 = Strongly disagree ... 5 = Strongly agree)

Source of the solution

“How much of your final solution resulted from your own attempts versus external assistance?”

(Slider: 0–100%)

Reflection on problem-solving strategy

“I reflected on my approach before and/or after solving the task.”

(1 = Strongly disagree ... 5 = Strongly agree)

Open-Ended Reflection Items

In addition to the structured questions, students respond to one or two open-ended prompts to capture qualitative insights into their learning process:

- “What did you learn from this exercise?” (free text)
- “What difficulties did you encounter while solving the task?” (free text)

These items provide a clear and implementable specification for integrating reflective elements into ACA while generating useful data for evaluating students’ learning strategies and AI tool usage.

4.3 Participants

Participants will be students enrolled in a programming course. Approximately 10–30 students are expected to participate, with 5–15 students assigned to each group depending on course size and availability. Participation will be coordinated with the course instructor.

4.4 Tasks

Participants will solve seven programming tasks of medium difficulty. The tasks are comparable in complexity to typical introductory programming exercises such as FizzBuzz, CSV-based data analysis, or simple object-oriented tasks (e.g., implementing a Vector2D class).

4.5 Data Collection

Data will be collected from multiple sources:

- **Quantitative data:** System logs from the ACA system, including time before first submission, number of submissions, solution correctness, and self-reported AI tool usage.
- **Qualitative data:** Semi-structured interviews conducted before and after the experiment to capture students' perceptions of AI tool usage and reflective prompts.

4.6 Measurability of Variables

To ensure traceability and objectivity, each variable in the study is operationalized using measurable indicators obtained either from the ACA system logs, automated code analysis tools, or self-reported survey items.

Table 1 summarizes how the main variables are measured in this study.

Aspect	Measurement
Solution quality	Automated metrics derived from static code analysis tools (e.g., <code>pylint</code> or <code>ESLint</code>), including style guideline adherence, code length, and indicators of structural complexity. These metrics are complemented by the pass/fail result provided by the ACA test suite.
Time before first submission	Measured using ACA system logs capturing the timestamp of the first submission for each task.
Number of submissions	Count of all submissions per task recorded in the ACA system logs.
First-attempt correctness	Binary indicator (pass/fail) based on whether the first submitted solution successfully passes the automated tests in ACA.
Self-reported AI usage	Measured through Likert-scale and slider items integrated into the reflection module (e.g., frequency of AI tool usage and degree of external assistance). Potential self-report bias is acknowledged as a limitation.

Table 1: Operationalization of study variables

In this study, *software quality* is operationalized through automated metrics that can be computed directly within the ACA environment. These include static analysis scores and automated test outcomes. Manual or instructor-based grading of code quality is explicitly excluded from the scope of this study to maintain objectivity and reproducibility.

Self-directed activity is approximated using behavioral indicators available in the system logs, including the time before the first submission, the number of submission attempts, and the students' self-reported reflections on their problem-solving process.

4.7 Analysis

Quantitative data from the ACA system logs will be analyzed to compare the control group and the experimental group. The analysis focuses on differences in:

- time before the first submission,
- number of submission attempts,
- first-attempt correctness, and
- self-reported AI tool usage.

Descriptive statistics and group comparisons will be used to identify potential differences between the two conditions.

Qualitative data obtained from interviews and open-ended reflection responses will be analyzed using thematic analysis in order to identify recurring patterns in students' learning strategies and perceptions of AI-assisted programming.

4.8 Ethics

Informed consent, anonymization, and ethical approval as required.

5 Related Work

Automated Code Assessment. Messer et al. (2024) conducted a systematic review of automated grading and feedback tools and found that most systems focus primarily on assessing correctness through unit testing, while feedback is often limited to pass/fail results or comparisons between expected and actual outputs. Comb  fis (2022) provides an overview of automated code assessment approaches used in education, classifying techniques and tools and discussing both static and dynamic assessment methods. Ihantola et al. (2010) analyze automatic assessment systems developed between 2006 and 2010 and highlight key system features such as automated tests, resubmission policies, and security mechanisms. Nayak et al. (2022) compare several automated assessment tools used in programming education, while Caiza and Del Alamo (2013) review existing tools and identify limitations such as the lack of a common grading model across systems.

AI and ChatGPT in Assessment. Recent research has also explored the use of AI-based systems for grading programming assignments. Jukiewicz (2024) evaluates the use of ChatGPT as a grading tool and reports a strong correlation between AI-generated grades and those assigned by human instructors. However, the study also highlights several limitations, including potential hallucinations, the need for teacher verification, and cost considerations. These limitations suggest that AI should complement rather than replace pedagogical approaches that support students' learning processes.

MOOCs and Formative Assessment. Pieterse (2013) discusses the role of automated assessment in large-scale online learning environments such as MOOCs and empha-

sizes the importance of formative assessment and opportunities for repeated practice with feedback.

Research Gap. Although automated code assessment systems are widely used in programming education, existing approaches primarily focus on evaluating correctness and efficiency of solutions. Mechanisms that promote reflection and self-regulated learning are rarely integrated into such systems. This thesis addresses this gap by extending an ACA system with a reflection module designed to encourage students to reflect on their learning strategies and their use of AI tools.

5.1 Annotated list of resources

Reference	Content	Personal Notes
Jukiewicz, 2024	ChatGPT as grading tool; advantages and limitations	Direct relevance for AI context; supports reflection-focused design
Messer et al., 2024	Systematic review of 121 papers on grading and feedback tools	Main overview; feedback limitations
Nayak et al., 2022	Review of automated assessment tools	Tool comparison, positioning of ACA

Table 2: Selected references with personal annotations.

6 Work Plan

Estimated Date	Milestone
Month 1	Literature review; finalize Exposé; ethics/consent; design reflection module
Month 2	Implement ACA reflection module (form, storage, integration); pilot test
Month 3	Recruit participants; run experiment; collect system logs; conduct interviews
Month 4	Data analysis; write thesis; revision and submission

Table 3: Planned milestones with detailed description of the results accomplished.

7 Preliminary thesis structure

1. Introduction
 - 1.1. Motivation and problem
 - 1.2. Research question and hypothesis
 - 1.3. Structure of the thesis
2. Theoretical and Methodological Foundation
 - 2.1. Related Work
 - 2.1.1. Automated code assessment
 - 2.1.2. AI in programming education
 - 2.1.3. Reflection and self-regulated learning
 - 2.2. Method
 - 2.2.1. Study design
 - 2.2.2. ACA reflection module (implementation)
 - 2.2.3. Participants and tasks
 - 2.2.4. Data collection and analysis
3. Results
 - 3.1. Quantitative results (logs, correctness, self-reports)
 - 3.2. Qualitative results (interviews)
4. Discussion
 - 4.1. Interpretation
 - 4.2. Limitations (e.g., self-report bias)
 - 4.3. Implications for practice
5. Conclusion and Outlook
6. References
7. Appendix (e.g., interview protocol, questionnaire items)

References

- [1] Combéfis, S. (2022). *Automated Code Assessment for Education: Review, Classification and Perspectives on Techniques and Tools*. *Software*, 1(1), 3–30. [doi:10.3390/software1010002](https://doi.org/10.3390/software1010002)
- [2] Caiza, J. C., & Del Alamo, J. M. (2013). *Programming Assignments Automatic Grading: Review of Tools and Implementations*. Proceedings of EDULEARN13, Barcelona, Spain.
- [3] Ihantola, P., Ahoniemi, T., Karavirta, V., & Seppälä, O. (2010). *Review of Recent Systems for Automatic Assessment of Programming Assignments*. Koli Calling '10, Koli, Finland. ACM.
- [4] Jukiewicz, M. (2024). *The future of grading programming assignments in education: The role of ChatGPT in automating the assessment and feedback process*. *Thinking Skills and Creativity*, 52, 101522. [doi:10.1016/j.tsc.2024.101522](https://doi.org/10.1016/j.tsc.2024.101522)
- [5] Messer, M., Brown, N. C. C., Kölling, M., & Shi, M. (2024). *Automated Grading and Feedback Tools for Programming Education: A Systematic Review*. *ACM Transactions on Computing Education*, 24(1), Article 10. [doi:10.1145/3636515](https://doi.org/10.1145/3636515)
- [6] Nayak, S., Agarwal, R., & Khatri, S. K. (2022). *Automated Assessment Tools for Grading of Programming Assignments: A Review*. 2022 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India.
- [7] Pieterse, V. (2013). *Automated Assessment of Programming Assignments*. CSERC '13, San Diego, CA, USA. [doi:10.1145/2556610.2556616](https://doi.org/10.1145/2556610.2556616)